

Ridge Regression: Regularisation and Intuition

Contents

1. Overview	2
2. The problem: overfitting in linear models	2
3. The idea: penalise large coefficients	3
4. The effect of λ	5
4.1 One feature	5
4.2 A flexible model	6
4.3 Many features: the diabetes data	7
5. Summary	9
6. Sources	9
7. Key terms	9

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Standardization ([Note 24](#)); Vector magnitude, distance and scalar operations ([Note 39](#)); Multiple linear regression ([Note 55](#)); Normal equation ([Note 55](#)); Gradient descent ([Note 57](#)); Overfitting ([Note 61](#)).
- **Leads to:** Lasso regression ([Note 67](#)); Elastic Net ([Note 69](#)); Support vector machines ([Note 92](#)); XGBoost ([Note 123](#)); Dropout ([Note 1024](#)); L1 and L2 regularisation in neural networks ([Note 1026](#)).
- **Compare with:** Lasso regression ([Note 67](#)); L1 and L2 regularisation in neural networks ([Note 1026](#)).

1. Overview

Key point: Regularisation adds a penalty to the loss so the model cannot overfit as easily. Ridge regression penalises the squares of the coefficients, which keeps them small.

Regularisation is a technique that adds extra information to a model to reduce overfitting. Regularisation sits next to bagging and boosting as one of the main tools against high variance, and it is especially important for linear models: linear regression, logistic regression and others.

There are three standard regularised versions of linear regression:

Name	Penalty	Also called
Ridge regression	sum of squared coefficients	L2 regularisation
Lasso regression	sum of absolute coefficients	L1 regularisation
Elastic Net	a mix of both	

This Note covers the idea of regularisation and of Ridge. The next three Notes derive Ridge mathematically, train it with gradient descent, and list its key properties; Lasso and Elastic Net follow.

2. The problem: overfitting in linear models

Key point: An overfitting linear model has extreme coefficients: a very steep line that chases the training points.

Overfitting means a model performs very well on the training data but poorly on new data, and gives very different results when trained on different samples (the high variance of the previous Note).

For linear regression, overfitting shows up in the coefficients. Each **feature** (an input variable, one column of the data table) gets one coefficient, and each **observation** (one record, one row of the table) is one training point. The **target** is the value we predict. Take the extreme case of only two training points. The least-squares line passes exactly through both, with zero training error. Its slope is whatever those two points dictate, which may be much steeper than the real pattern. On new points the line can be far off.

Figure 1 (left) draws 20 such samples of two points, all from the same gentle pattern (the dashed line). Watch how far the least-squares lines swing: from one sample to the next the slope ranges from -2.0 to 2.3 , while the true slope is 0.9 . The right panel previews the fix of Section 3: the Ridge slopes stay between 0 and 0.7 .

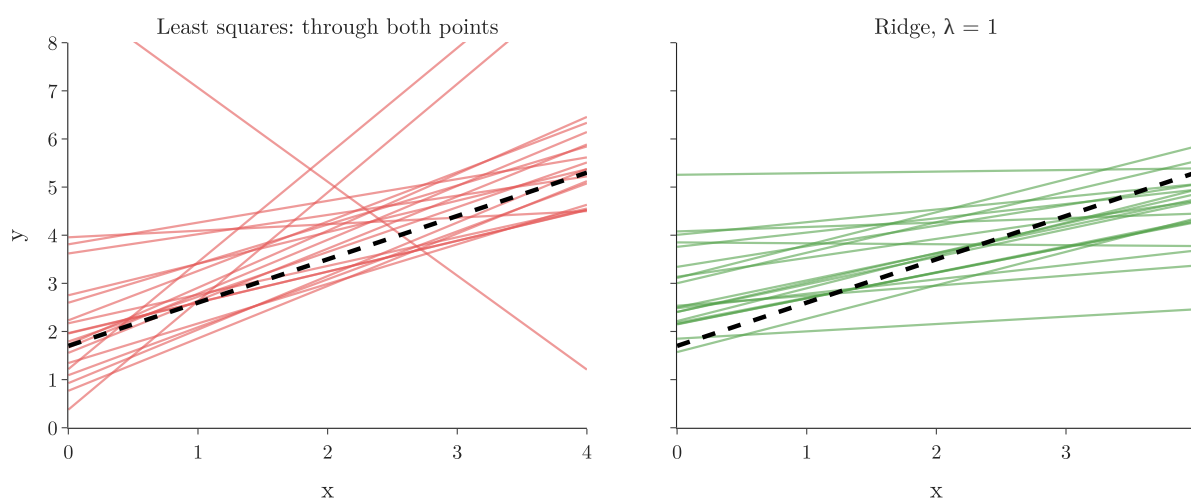


Figure 1: 20 samples of two points from the same pattern (dashed). Left: the least-squares line through each pair. Right: Ridge with $\lambda = 1$ on the same pairs; the lines stay close together but lean flatter than the pattern

So the warning sign of an overfitting linear model is **large coefficients**: the line or plane tilts sharply to fit the training points exactly.

3. The idea: penalise large coefficients

Key point: Ridge minimises the usual squared error plus λ times the squared slope. A steep line now costs something, so the model prefers a flatter one unless the data strongly supports the steep one.

Ridge regression changes the loss function. In words: the loss is the usual sum of squared errors, plus a penalty equal to λ times the square of the slope.

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda m^2$$

λ (lambda) is a hyperparameter, at least 0, that sets how strong the penalty is. The intercept b is not penalised: it only measures the average level of y , and shrinking it would not make the line flatter (ISL §6.2.1).

With numbers, Figure 2 has two training points, $(1, 2)$ and $(3, 5)$, and $\lambda = 1$.

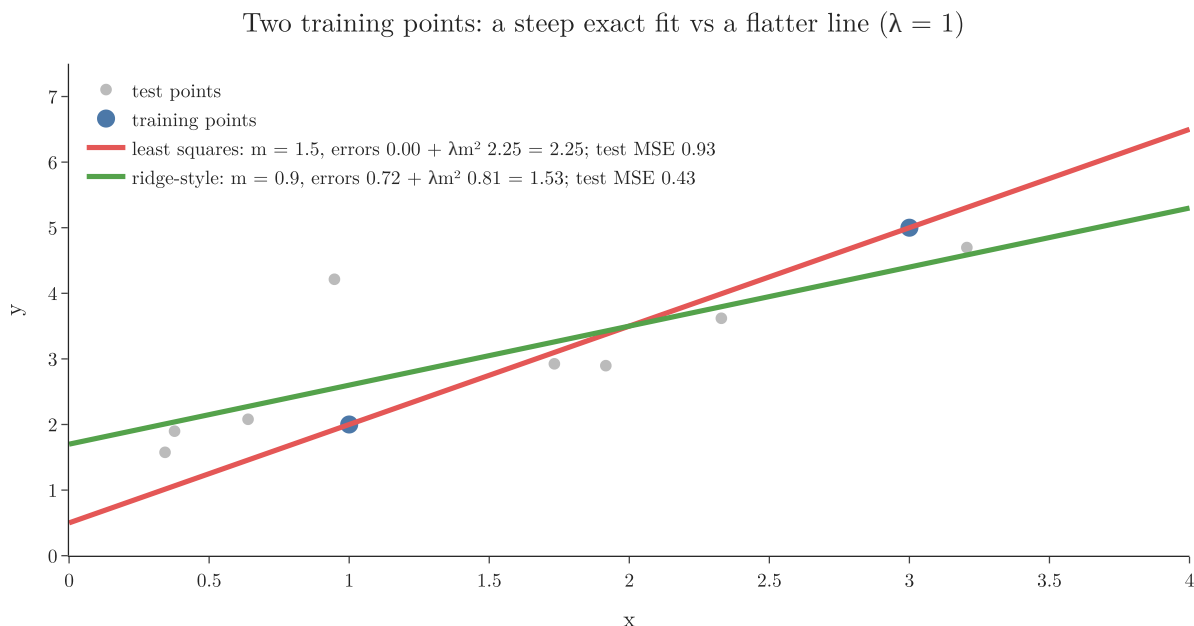


Figure 2: Two training points: the exact fit vs a flatter line

- **The least-squares line** passes through both points: slope 1.5, errors 0. With the penalty, its loss is $0 + 1 \times 1.5^2 = 2.25$.
- **A flatter line** with slope 0.9 misses both points slightly: errors 0.72. Its loss is $0.72 + 1 \times 0.9^2 = 1.53$.

With the penalty, the flatter line has the lower loss, so Ridge prefers it. Figure 2 is a made-up example: its grey test points were drawn from a flatter pattern, to picture the case Ridge is built for. Section 4.3 tests the idea on real data.

Figure 3 lets λ grow on the same two points. Watch the line pivot about the middle point $(2, 3.5)$ and flatten, while the bars trade off: the errors grow, and the penalty first grows and then shrinks as the slope nears 0. At $\lambda = 1$ the best line has slope 1.0 and loss $0.50 + 1.00 = 1.50$, a little below the 1.53 of the slope-0.9 line.

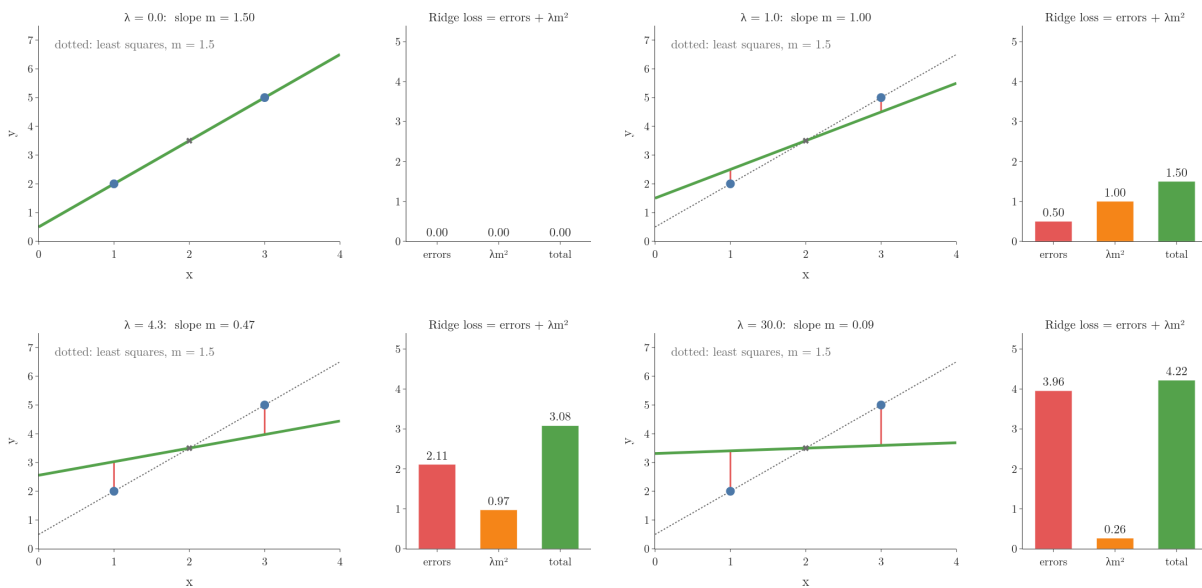


Figure 3: λ grows from 0 to 30 on the two training points. The green line is the Ridge fit (dotted: least squares); red sticks are its errors. The bars show the squared errors, the penalty λm^2 and their sum. After StatQuest’s “Regularization Part 1: Ridge (L2) Regression” (Starmer)

Ridge gives up a little accuracy on the training data (some bias) in exchange for a model that changes less from sample to sample (less variance). As λ grows, variance falls and bias rises (ISL §6.2.1).

Think of a tailor who fits a suit to one photo of a customer. A suit that follows every fold of that one photo fits badly on the real day. A tailor who keeps the cut a little plain, trusting the photo less, does better on average. Ridge keeps the coefficients plain in the same way.

4. The effect of λ

Key point: $\lambda = 0$ is ordinary linear regression. As λ grows, the coefficients shrink towards 0. Too large a λ flattens the model into underfitting.

4.1 One feature

Key point: With one feature, a larger penalty gives a flatter line.

Figure 4 (left) fits 100 observations with one feature. In scikit-learn the penalty strength is called **alpha** instead of λ .

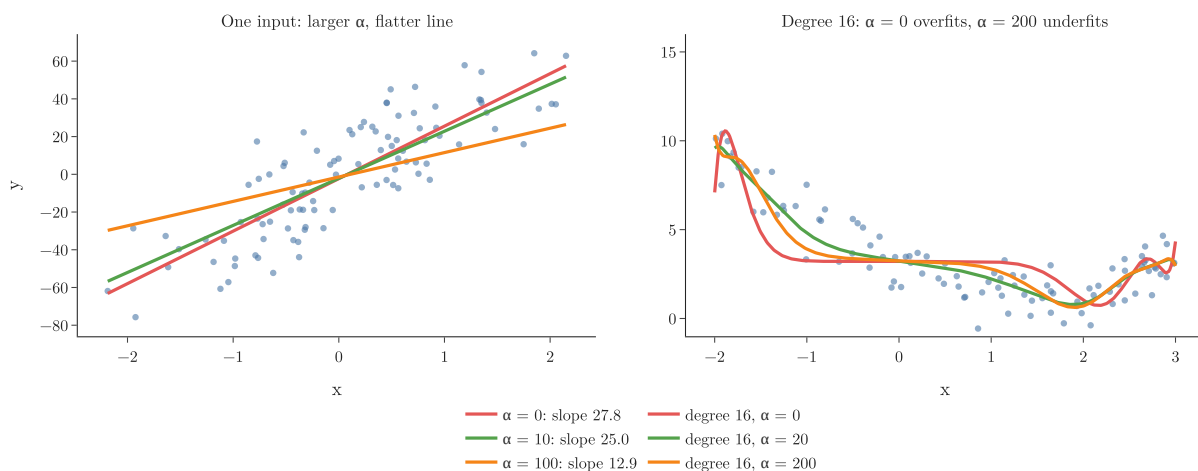


Figure 4: The effect of alpha with one feature and with a degree-16 polynomial

alpha	Slope
0 (linear regression)	27.8
10	25.0
100	12.9

The slope shrinks as alpha grows, while the intercept changes little.

4.2 A flexible model

Key point: On a degree-16 polynomial, $\alpha = 0$ overfits, $\alpha = 20$ follows the pattern, $\alpha = 200$ is too stiff.

Regularisation matters most for flexible models. Figure 4 (right) fits a degree-16 polynomial to curved data:

- **alpha = 0:** the curve bends to chase individual points and swings at the edges: overfitting.
- **alpha = 20:** the penalty keeps the many coefficients small, and the curve follows the overall pattern.
- **alpha = 200:** the coefficients are squeezed so hard that the curve is too flat in places: underfitting.

The best alpha is somewhere in between and is found by trying values on held-out data, like any hyperparameter.

4.3 Many features: the diabetes data

Key point: As alpha grows, every coefficient is pulled towards 0. With few training observations, a small alpha raises test R^2 a lot; a huge alpha destroys the model.

Figure 5 trains Ridge on the diabetes data (10 features, 442 observations; the target is disease progression one year later) with alpha from 0.0001 to 100,000.

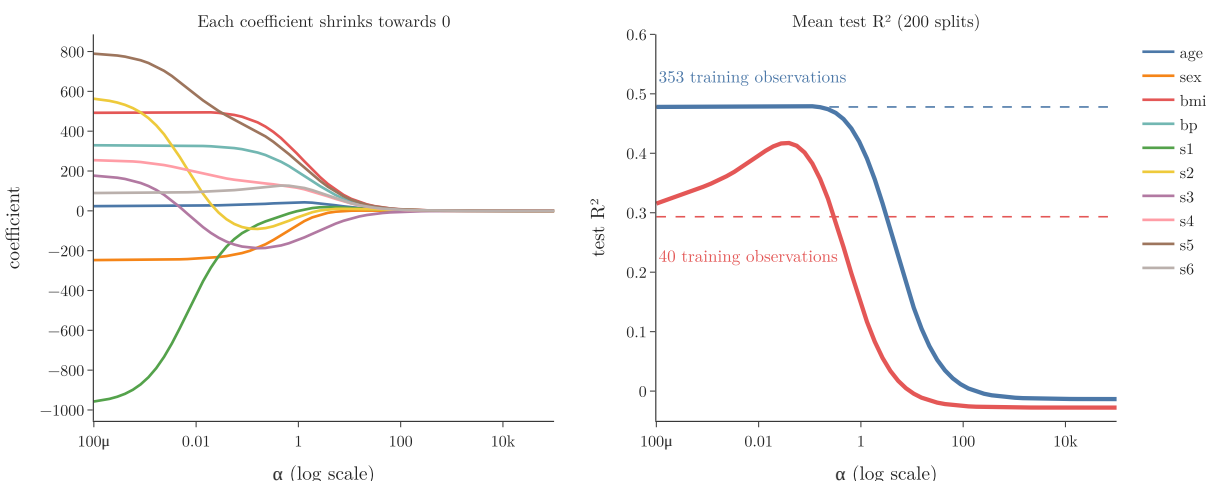


Figure 5: Coefficients and test R^2 against alpha on the diabetes data

- **Left** (one split, 353 training observations): at a large alpha every coefficient ends near 0, though not always in a straight line (age and s6 first grow, s3 changes sign). The biggest one, s1 (−971), shrinks fastest. Why was it so big? s1 and s2 are strongly correlated ($r = 0.90$). Plain linear regression gives such a pair large opposite coefficients (−971 and +574) that partly cancel each other, and Ridge’s limit on coefficient size stops this (ESL §3.4.1). At alpha 0.01 the pair is already down to −421 and +138.
- **Right** (mean test R^2 over 200 random splits; dashed lines are plain linear regression):
 - **40 training observations:** plain linear regression scores 0.29. Ridge at alpha 0.04 scores 0.42, a large gain.
 - **353 training observations:** 0.478 against 0.480, almost no gain.
 - **Huge alpha:** with alpha 100,000 every coefficient is about 0 and $R^2 \approx 0$: the model just predicts the average.

Why the difference? With 40 observations and 10 features, the least-squares coefficients swing a lot from sample to sample (high variance), so taming them pays off. With 353 observations the least-squares fit is already stable, so there is little variance to remove. Ridge works best where least squares has high variance (ISL §6.2.1).

Python: Ridge in scikit-learn.

```
from sklearn.linear_model import Ridge
```

```
ridge = Ridge(alpha=0.1)           # alpha is the  $\lambda$  of the formula
ridge.fit(X_train, y_train)
ridge.coef_                         # smaller than LinearRegression's
ridge.score(X_test, y_test)
```

Extra: The penalty depends on the size of each coefficient, and coefficients depend on the scale of their features. A feature measured in grams gets a much smaller coefficient than the same feature in kilograms, so it would be penalised less. For this reason features are standardised before Ridge (ISL §6.2.1; ESL §3.4.1). The diabetes features already come scaled (scikit-learn docs, `load_diabetes`). In a pipeline: `make_pipeline(StandardScaler(), Ridge(alpha=1))`.

5. Summary

alpha (λ)	Effect
0	ordinary linear regression
small	smaller coefficients; better test R^2 when training data is small
large	coefficients near 0, the model underfits

- Overfitting linear models have extreme coefficients.
- Ridge adds $\lambda \sum \beta_j^2$ to the loss; the intercept is not penalised.
- Ridge trades a little bias for less variance, and helps most where least squares has high variance.
- alpha is tuned on held-out data; features should be standardised first.

6. Sources

Built from

- CampusX, “Ridge Regression Part 1 | Geometric Intuition and Code | Regularized Linear Models”, YouTube, <https://www.youtube.com/watch?v=aEow1QoTLo0>
- Starmer, J. “Regularization Part 1: Ridge (L2) Regression.” statquest.org. The idea of animating the line as λ grows (Figure 3).

Other references

- **ISL**: James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning*, 2nd ed. Springer, 2021. Section 6.2.1, pp. 237–241.
- **ESL**: Hastie, T., Tibshirani, R. and Friedman, J. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. Section 3.4.1, pp. 63–64.
- **scikit-learn docs**: `sklearn.datasets.load_diabetes`, scikit-learn 1.9 documentation.

7. Key terms

Term	Meaning
Regularisation	Adding a penalty to a model’s loss to reduce overfitting
Ridge regression	Linear regression with a penalty on the sum of squared coefficients
L2 regularisation	Another name for the squared-coefficient penalty used by Ridge
Lasso regression	Linear regression with a penalty on the sum of absolute coefficients (L1)
Elastic Net	Linear regression with a mix of the L1 and L2 penalties

Term	Meaning
λ (lambda), alpha	The strength of the regularisation penalty; alpha in scikit-learn
Shrinkage	The pulling of coefficients towards 0 by a penalty
Feature, target, observation	An input variable (column); the value predicted; one record (row)